

Cross-System Data Drift & Trust Monitoring Platform

Project Report

1. Introduction

Organizations typically operate multiple systems — such as CRM, Billing, and Analytics — that are maintained independently. Because these systems are updated separately, they often drift out of sync: a customer may exist in one system but not another, or the same business metric (like revenue) may be reported differently across systems. Such inconsistencies usually go unnoticed until they cause a real business problem.

For this project, I built a data pipeline on Databricks that ingests data from CRM, Billing, and Analytics sources, cleans it, cross-verifies records between systems, detects drift, and calculates a single Trust Score to represent how reliable the data is at any given point. Below is a breakdown of exactly what I did, step by step.

2. Objective

- Bring CRM, Billing, and Analytics data into one place and clean it
- Compare records across systems to find missing or mismatched data
- Detect drift — changes in data volume, schema, or distribution over time
- Calculate one Trust Score that reflects overall data reliability
- Present the findings through SQL reports and a Databricks dashboard

3. Tools & Technologies Used

Technology	Purpose
Python	Core data processing logic
PySpark	Distributed data processing, cleaning, and transformation
SQL	Aggregations and queries powering the dashboard
Delta Lake	Storage layer for Bronze/Silver/Gold tables
Databricks	Data engineering platform — notebooks, tables, and dashboard

4. Dataset Description

Before getting into the implementation, here's a quick look at the three datasets the pipeline works with and the fields each one contains.

CRM Dataset

- Customer ID
- Name
- Email
- City
- Signup Date

Billing Dataset

- Customer ID
- Transaction ID
- Amount
- Status
- Transaction Date

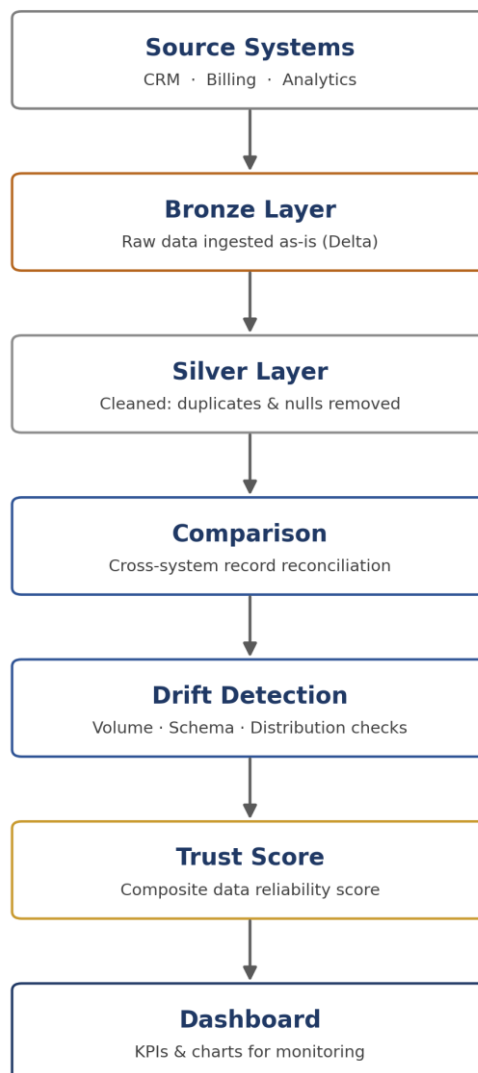
Analytics Dataset

- Date
- Total Customers
- Total Revenue
- Average Transaction

CRM and Billing are record-level datasets (one row per customer / transaction), while Analytics is a pre-aggregated, summary-level dataset — which is exactly why it's used as an independent cross-check against Billing's own revenue totals.

5. Architecture

The pipeline moves data through a series of stages, each one building on the output of the previous stage, ending in a dashboard that's used for ongoing monitoring:



Each arrow in this flow corresponds to one notebook in the project — for example, the move from Bronze to Silver is handled entirely inside 02_Data_Cleaning.ipynb, and the move from Trust Score to Dashboard is handled by the SQL queries in 06_SQL_Analysis.ipynb.

6. Work Done — Step by Step

I organized the project into six notebooks, following a Bronze → Silver → Gold (Medallion) architecture, where each layer builds on the previous one.

Step 1: Data Ingestion (01_Data_Ingestion.ipynb)

I started by reading the raw CRM, Billing, and Analytics tables and writing them, unchanged, into Delta tables — this became my Bronze layer. This gave me a stable, versioned starting point before any cleaning.

- CRM: 10,710 records ingested
- Billing: 11,914 records ingested
- Analytics: 912 records ingested

Step 2: Data Cleaning (02_Data_Cleaning.ipynb)

Next, I checked each Bronze table for missing values and duplicate rows before cleaning anything, so I had a clear before/after picture.

- CRM: 321 rows with missing email, 194 duplicate rows
- Billing: 238 rows with missing transaction amount, 222 duplicate rows
- Analytics: 18 rows with missing revenue figure, 0 duplicates

After removing the nulls and duplicates, I wrote the cleaned data out as the Silver layer:

- CRM: 10,195 records
- Billing: 11,454 records
- Analytics: 894 records

Step 3: Cross-System Comparison (03_Data_Comparison.ipynb)

Using the cleaned Silver tables, I compared CRM and Billing against each other using joins on customer_id, and also compared the revenue figures reported by Billing and Analytics.

- **CRM → Billing gap** — 3,969 customers exist in CRM but have no matching record in Billing
- **Billing → CRM gap** — 1,457 customers exist in Billing but have no matching record in CRM
- **No transactions** — 3,911 CRM customers have zero recorded transactions in Billing
- **Revenue mismatch** — Billing reported ₹60,03,866.28 vs. Analytics' ₹43,91,369.72 — a gap of ₹16,12,496.56

This step is what showed me that the systems don't fully agree with each other, even after cleaning — the gaps are real and worth flagging.

Step 4: Drift Detection (04_Drift_detection.ipynb)

Here I built three checks to detect if something in the data was drifting away from what's expected, each compared against a threshold I set:

- **Volume Drift** — % difference in record counts between CRM and Billing (threshold: 10%)
- **Schema Drift** — whether column names/types still match between systems
- **Distribution Drift** — whether the average billing transaction has moved beyond ₹500

Results:

- Volume Drift came out to 11.18% → Alert Raised
- Schema Drift was detected → Alert Raised
- Average billing amount was ₹513.50, above the ₹500 threshold → Alert Raised

Step 5: Data Trust Score (05_Trust_Score.ipynb)

To make all these findings easy to understand at a glance, I combined missing-value percentage and volume drift into a single score:

$$\text{Trust Score} = 100 - \text{Missing Value \%} - \text{Volume Drift \%}$$

- **Score Bands** — Excellent (≥ 90) · Good (≥ 75) · Average (≥ 60) · Poor (< 60)
 - Total records evaluated: 22,543
 - Missing values after cleaning: 0 → 0.0%
 - Volume drift: 12.35%
 - Final Trust Score: 87.65 / 100 — rated "Good"

This confirmed that missing data wasn't the issue anymore after cleaning — the score is being pulled down mainly by the volume mismatch between CRM and Billing, which matches what I found in the comparison step.

Step 6: SQL Analysis (06_SQL_Analysis.ipynb)

In the last notebook, I wrote SQL queries directly against the Silver and Gold tables to generate the numbers needed for reporting — record counts, revenue trends, status breakdowns, and city-wise customer distribution.

- **Transaction Status** — Completed 9,085 · Pending 1,203 · Failed 832 · Refunded 572
- **Top Cities by Customers** — Mumbai (800), Bangalore (743), Delhi (731), followed by Kanpur, Patna, Indore, Lucknow, Coimbatore, Ahmedabad, and Kolkata

These queries are exactly what feed the dashboard, described in Section 8. Note: since the datasets are synthetically generated, exact counts shift slightly each time the data is regenerated, but the overall pattern — around 11–12% volume drift and a consistent "Good" trust rating — stayed the same across runs.

7. Delta Lake Implementation

I used Delta Lake as the storage format for every layer (Bronze, Silver, Gold) instead of plain CSV files, for a few concrete reasons:

- **ACID Transactions** — every write (saveAsTable, overwrite, merge) is atomic — a failed or partial job can't leave a table in a half-written, corrupted state, which plain CSV writes don't guarantee
- **Version History** — Delta automatically keeps a full history of every write to a table, so nothing is ever silently overwritten and lost
- **DESCRIBE HISTORY** — I used this command after every major write (Bronze, Silver, and Gold tables) to confirm what changed, when, and by which operation — this was genuinely useful for debugging while building the pipeline
- **CREATE OR REPLACE / mode("overwrite")** — used when rewriting Bronze/Silver/Gold tables on each run, while still preserving prior versions underneath for rollback or time-travel if needed
- **Schema Enforcement** — Delta rejects writes that don't match a table's expected schema, which adds an extra layer of protection against silently corrupt data — something CSV has no concept of

In short: CSV would have worked for a one-time script, but this pipeline is meant to be re-run repeatedly on refreshed data, and Delta Lake is what makes that safe and auditable across Bronze, Silver, and Gold.

8. Dashboard

I built a Databricks dashboard on top of the Gold-layer tables (`comparison_report`, `drift_report`, `trust_score_gold`) so that the results of the pipeline are visible at a glance, without needing to open any notebook. It's organized into two parts — KPI cards up top for a quick status check, and charts below for deeper detail.

KPI Cards

- **CRM Records** — shows the current Silver-layer CRM count, so any sudden change in record count is immediately visible
- **Billing Records** — same idea for Billing — a quick sanity check on data volume
- **Analytics Records** — keeps the smallest, summary-level dataset visible alongside the other two
- **Trust Score** — the single most important number on the dashboard — shows the current score and its rating (Excellent/Good/Average/Poor) so anyone can tell the overall data health in one glance

Charts

- **Revenue Trend** — monthly revenue from Billing over time — included to catch any sudden, unexpected jump or drop in revenue
- **Customer Trend** — how the customer base is growing over time, based on CRM signup dates
- **Revenue by Status** — revenue split across completed / pending / failed / refunded transactions — useful for spotting if failed or refunded transactions are eating into revenue
- **Billing Status Breakdown** — count of transactions by status — a quick way to see if failure/refund rates are unusually high
- **Customers by City** — geographic distribution of customers — mainly included to sanity-check that customer data looks realistic and isn't skewed by a data-generation or ingestion bug

Each visualization was chosen to answer one specific question a reviewer might ask — "has anything changed?", "can I trust this data right now?", "where is revenue or customer volume coming from?" — rather than just displaying data for its own sake.

9. Challenges I Faced

- Working with large synthetic datasets efficiently across multiple joins in PySpark
- Designing a schema-drift check that's meaningful when systems have different schemas by design
- Deciding on sensible thresholds for volume and distribution drift alerts
- Reconciling revenue numbers that were reported differently across Billing and Analytics

10. What I Would Improve Next

- Track a single table's schema over time (using Delta Lake time-travel) instead of comparing two different systems directly
- Move from batch processing to real-time monitoring using Spark Structured Streaming
- Replace fixed thresholds with statistical or ML-based anomaly detection
- Add automated alerting (email/Slack) and scheduled runs instead of manual execution

11. Conclusion

Through this project, I built a complete pipeline — from raw data ingestion to a final Trust Score — that can catch real, cross-system data issues automatically. It found genuine problems in the data: thousands of customers missing between CRM and Billing, a revenue mismatch of over ₹16 lakh, and a volume drift high enough to trigger an alert. All of this was summarized into one Trust Score (87.65, rated Good), making it easy to understand the overall health of the data without digging through individual tables. The Databricks dashboard

makes these results visible on an ongoing basis, and I've laid out clear next steps to make the pipeline more robust going forward.